



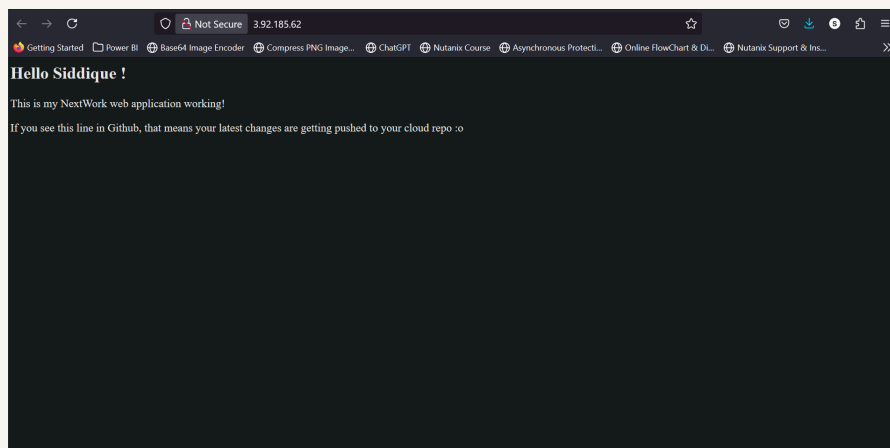
nextwork.org

Deploy a Web App with CodeDeploy



Muhammad Sadique

<https://www.linkedin.com/in/muhammad-sadique->





Introducing Today's Project!

In this project i will demonstrate deployment of my build package from staging environment into production environment using AWSDeploy which is a Continuous Deployment Service. I am doing this project as part of CI/CD Pipeline project to help understand how different pieces of CI/CD Pipeline fits together for rapid and short deployment of Dev Code.

Key tools and concepts

Services I used was AWS CodeDeploy. Key concepts i learnt include Deployment group, CodeDeploy Agent, Deployment strategies etc.

Project reflection

This project took me approximately 8 hours. The most challenging part was to troubleshoot the deployment. It was most rewarding to see the troubleshoot actually worked.

This project is part five of a series of DevOps projects where I'm building a CI/CD pipeline! I'll be working on the next project tomorrow inshaALLAH.

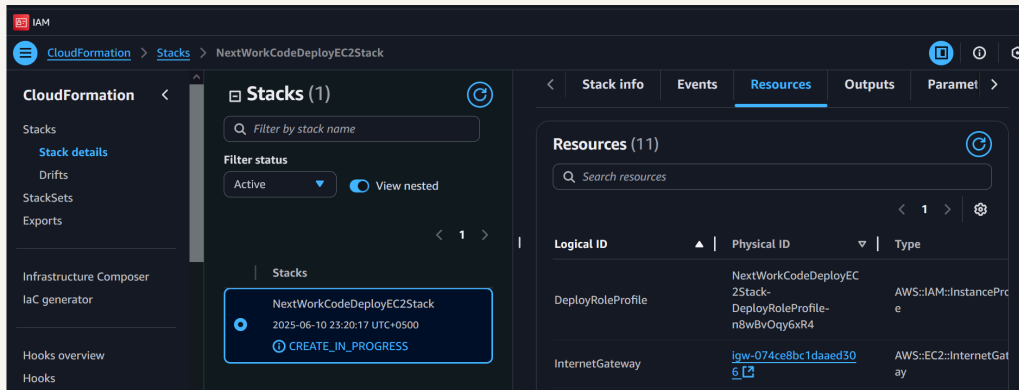


Deployment Environment

To set up for CodeDeploy, I launched an EC2 instance and VPC because i wanted a separate EC2 instance for my production environment.

Instead of launching these resources manually, I used AWS CloudFormation Stack. When I need to delete these resources i'd just delete the stack rather than deleting all resources one by one.

Other resources created in this template include: 1. VPN 2. Subnets 3. Route Tables 4. Internet Gateway 5. Web Server (EC2 Instance) They are also in the template because we needed to create an isolated production environment without any dependency or shared resource with Dev environment.





Deployment Scripts

Scripts are like mini-programs that automate tasks. They're essentially text files filled with commands that you'd normally type one by one in the terminal, but packaged together so they run automatically in sequence. To set up CodeDeploy, I also wrote scripts to automate deployment tasks.

The `install_dependencies.sh` script sets up all the software needed to run our website by installing programs (called Tomcat and Apache) that handle internet traffic and host our application. It then creates special settings that let these programs to work together, making our website accessible to visitors on the internet.

`start_server.sh` will start Apache and Tomcat services

This `stop_server.sh` script safely stops web server services by first checking if they're running. It uses `pgrep` to check for running processes of Apache (`httpd`) and Tomcat, and only attempts to stop the services if they are actually active. This prevents errors that could occur from trying to stop services that aren't running.

appspec.yml

Then, I wrote an appspec.yml file to tells CodeDeploy what to do to deploy your app, and when to do each step. The key sections in appspec.yml are version, os, files and hooks.

I also updated buildspec.yml to include appspec.yml and the scripts folder. The artifacts section we just edited is particularly important - it's telling CodeBuild: "After you build the app, make sure to include these additional files in the final package." We're adding our appspec.yml and scripts folder because CodeDeploy will need them to properly deploy the application.

```
! appspec.yml
1 version: 0.0
2 os: linux
3 files:
4   - source: /target/nextwork-web-project.war
5     destination: /usr/share/tomcat/webapps/
6 hooks:
7   BeforeInstall:
8     - location: scripts/install_dependencies.sh
9       timeout: 300
10      runas: root
11   ApplicationStart:
12     - location: scripts/start_server.sh
13       timeout: 300
14       runas: root
15   ApplicationStop:
16     - location: scripts/stop_server.sh
17       timeout: 300
18       runas: root
```

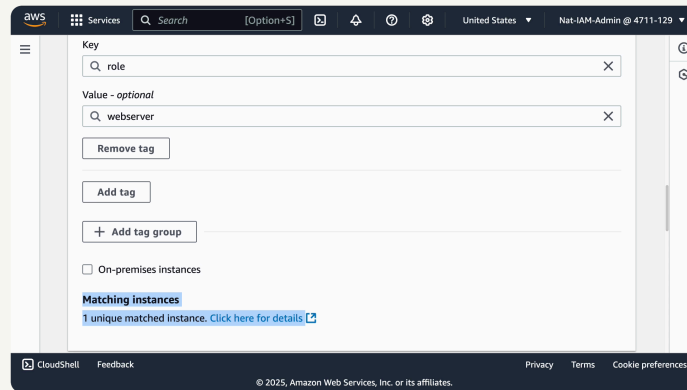


Setting Up CodeDeploy

A deployment group is a collection of EC2 instances that are grouped to deploy something together. A CodeDeploy application is the application that you'd like to deploy on one or many deployment groups.

To set up a deployment group, I also need to create an IAM role for CodeDeploy to grant permissions to access and manage AWS resources on my behalf. These permissions let CodeDeploy do things like: 1. Accessing EC2 instances to deploy applications. 2. Reading application artifacts from S3 buckets. 3. Updating Auto Scaling groups. 4. Write CloudWatch logs about what it's doing

We are using tags here so that CodeDeploy can identify the target EC2 instances for deployment. In our CloudFormation template, we tagged our EC2 instance with role: webserver. CodeDeploy will use this tag to find and deploy to the correct instance. Tags help us be super efficient for three reasons: Flexibility: If you add new instances with the same tag, CodeDeploy automatically includes them in future deployments. Self-documentation: Tags like role: webserver clearly explain what the instance does, making our AWS environment easier to understand. Integration: The CloudFormation template we used earlier already tagged our EC2 instance with role: webserver, so everything works together seamlessly.

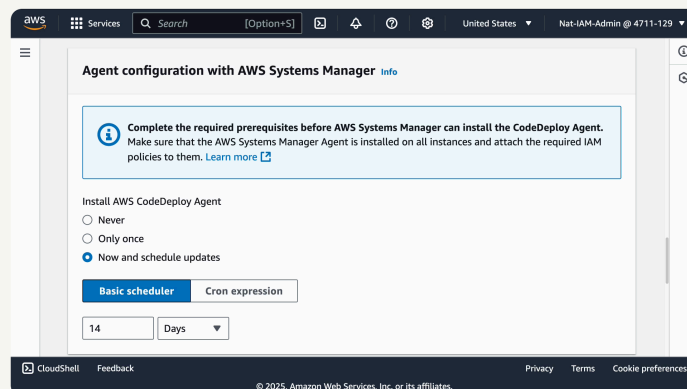




Deployment configurations

Another key settings is the deployment configuration, which affects rollout of our application. I used `CodeDeployDefault.AllAtOnce`, so that the new application gets deployed fast on all EC2 instances in the newly created deployment group.

The CodeDeploy Agent is a software that lets your EC2 instance communicate with CodeDeploy. Whenever you initiate a deployment, it's the CodeDeploy Agent that receives the deployment instructions (i.e. bash scripts) from CodeDeploy and carries them out on the EC2 instance.



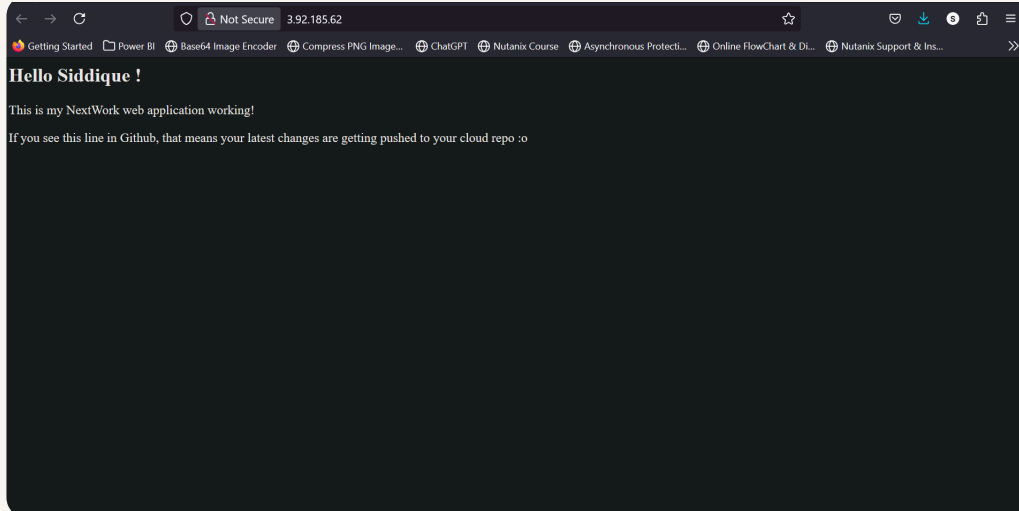


Success!

A CodeDeploy deployment is a single update to our application with its own ID and history. The difference to a deployment group is that: 1. Deployment group represents a collection of EC2 instances that are grouped to deploy something together. 2. We can have multiple deployment groups within the same application - maybe one for your test environment, another for staging, and a third for production.

I had to configure a revision location, which means that CodeDeploy will look into this location for my application build artifacts.

To check that the deployment was a success, I opened the DNS address or Public IP in my browser and there it was - my web application deployed on production ec2 instance that i early created using CloudFormation stack.





Disaster Recovery

In a project extension, I decided to introduce syntax error in `stop_server.sh` script. This will cause the deployment to fail because of syntax error.

I also enabled rollbacks with this deployment, which means that in case of error during deployments, i can rollback to last functioning configuration of my web application.

When my deployment failed, the automatic rollback didn't rollback to last working build artifact, because the AWS CodeDeploy rollback work in a way that i points to the latest artifact in S3 bucket which also contains the broken `stop_server.sh`. To actually recover from broken script i'd have to fix the errors in the script manually. In production environments, true automated rollbacks are often implemented with more advanced tools. AWS CodePipeline can be configured to automatically roll back to the last successful deployment when a failure is detected, which eliminates the need for manual intervention.



Developer Tools > CodeDeploy > Deployments

Deployment history  [View details](#) [Actions](#) [Copy deployment](#) [Retry deployment](#)

< 1 > 

	Deployment Id	Status	Deploym...	Compute ...	Application	Deploym...	Revision L...	Initiating event
☐	d-1XUBBWVAD	 Failed	In-place	EC2/On-p...	network-...	network-...	s3://next...	CodeDeploy rollback
☐	d-2TW6PJWAD	 Failed	In-place	EC2/On-p...	network-...	network-...	s3://next...	User action
☐	d-JYJ9P1WAD	 Failed	In-place	EC2/On-p...	network-...	network-...	s3://next...	CodeDeploy rollback
☐	d-FKX68DWAD	 Failed	In-place	EC2/On-p...	network-...	network-...	s3://next...	User action
☐	d-21978CWAD	 Succeeded	In-place	EC2/On-p...	network-...	network-...	s3://next...	User action



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

